

React Interview Questions

1. What is Reactjs?

Why React is called a JavaScript Library and not a Framework

Short answer (for interviews)

React is a JavaScript library because it focuses only on the UI (view layer) and does not enforce a complete application structure or rules like a framework does.

What is a Library?

A library:

- Solves a **specific problem**
- You **call the library**
- Gives you **freedom** to choose other tools

 **Example:**

- You decide **when and how** to use it
 - No strict folder structure
 - No fixed rules
-

What is a Framework?

A framework:

- Provides a **complete solution**
- **Calls your code** (inversion of control)
- Enforces **rules, structure, and patterns**

 **Example:**

- Fixed folder structure
- Built-in routing, state, HTTP, etc.
- You must follow its way

Why React is a Library

1 React only handles the UI (View Layer)

React is mainly responsible for:

- Creating UI components
- Rendering data to the DOM
- Managing component state

 React **does NOT** provide:

- Routing (React Router is separate)
- HTTP requests (Axios / Fetch)

- Global state management (Redux, Zustand)
- Authentication
- Backend communication rules

You must **choose these yourself**.

2 No strict project structure

React does **not force**:

- Folder structure
- File naming
- Architectural pattern

You decide:

```
components/  
pages/  
hooks/  
utils/
```

3 You control the flow (not React)

In React:

- **You call React**
- You decide when to render components
- You decide how to manage state

In frameworks:

- The framework controls execution
-

4 React can be used inside other frameworks

React is flexible and can be embedded in:

- Next.js
- Gatsby
- Remix

 These are **frameworks built on top of React**.

Then why people *think* React is a framework? 🤔

Because modern React apps use:

- Routing
- State management
- API handling
- Build tools

But these come from **external libraries**, not React itself.

Simple comparison table

Feature	React	Framework (Angular)
Type	Library	Framework
UI Handling	Yes	Yes
Routing		
State Management		
Project Structure		
Freedom	High	Low
Opinionated	No	Yes

Interview-ready answer (memorize this 🙌)

React is called a JavaScript library because it focuses only on building user interfaces and gives developers freedom to choose tools for routing, state management, and other features. A framework provides a complete, opinionated structure, which React does not.

2.Does React use HTML?

What is JSX?

JSX (JavaScript XML) is a **syntax extension of JavaScript** used in React that allows us to write **HTML-like code inside JavaScript**.

📌 It makes UI code **more readable, expressive, and easier to understand**.

Why do we use JSX?

JSX is used because:

- It makes UI code **simple and readable**
- It combines **logic + UI in one place**
- It helps React understand **what the UI should look like**

Without JSX, writing UI would be complex and verbose.

Example

JSX code

```
const element = <h1>Hello, React!</h1>;
```

What actually happens (behind the scenes)

JSX is **not understood by browsers**.

It is converted into JavaScript by **Babel**:

```
const element = React.createElement("h1", null, "Hello,  
React!");
```

📌 So **JSX** is just syntactic sugar for `React.createElement()`.

Key Features of JSX

❶ JSX looks like HTML but is JavaScript

- You can write tags
- You can use JavaScript expressions

```
const name = "Vishal";  
<h1>Hello, {name}</h1>
```

❷ You can use JavaScript inside JSX using `{ }`

```
<h2>{10 + 20}</h2>  
<h2>{isLoggedIn ? "Welcome" : "Please Login"}</h2>
```

❸ JSX must have one parent element

❌ Invalid:

```
<h1>Hello</h1>  
<p>World</p>
```

✅ Valid:

```
<>  
  <h1>Hello</h1>  
  <p>World</p>  
</>
```

(React Fragment)

4 JSX uses `className` instead of `class`

Because `class` is a JavaScript keyword:

```
<div className="container"></div>
```

5 JSX prevents injection attacks

JSX **automatically escapes values**, making apps **more secure** against XSS attacks.

Is JSX mandatory in React?

 No

React works without JSX, but JSX makes development **faster and cleaner**.

JSX vs HTML (Quick Comparison)

JSX	HTML
JavaScript based	Markup only
<code>className</code>	<code>class</code>
JS expressions allowed	Not allowed
Compiled by Babel	Directly run by browser

Interview one-line answer ★

JSX is a syntax extension for JavaScript used in React that allows writing HTML-like code inside JavaScript. It is compiled by Babel into `React.createElement()` calls.

Bonus Interview Question (with answer)

Q: Why React uses JSX instead of templates?

👉 Because JSX keeps **UI and logic together**, improving readability and maintainability.

If you want, I can:

- Give **JSX rules list**
- Provide **common JSX mistakes**
- Do **mock interview Q&A**
- Explain JSX in **simple Hinglish**

Just say 👍

yes

Great 👍

Let's do **JSX Mock Interview Q&A + Common Mistakes** (very useful for placements).

◆ Mock Interview: JSX Questions & Answers

Q1. What is JSX?

Answer:

JSX is a JavaScript syntax extension used in React that allows writing HTML-like code inside JavaScript. It is compiled by Babel into `React.createElement()`.

Q2. Is JSX mandatory in React?**Answer:**

No, JSX is not mandatory. React can work without JSX, but JSX makes the code more readable and easier to write.

Q3. Can browsers understand JSX directly?**Answer:**

No, browsers cannot understand JSX. It is converted into JavaScript by Babel before execution.

Q4. Why do we use `className` instead of `class` in JSX?**Answer:**

Because `class` is a reserved keyword in JavaScript, so React uses `className` to avoid conflicts.

Q5. How can we write JavaScript inside JSX?**Answer:**

By using curly braces `{ }` inside JSX.

Example:

```
<h1>{userName}</h1>
```

Q6. Why must JSX return a single parent element?

Answer:

Because JSX is converted into a single `React.createElement()` call, so it must return one root element. We can use `div` or `React.Fragment (<> </>)`.

Q7. What is React Fragment?

Answer:

React Fragment allows grouping multiple elements without adding an extra node to the DOM.

```
<>
  <h1>Hello</h1>
  <p>World</p>
</>
```

Q8. Is JSX secure?

Answer:

Yes, JSX escapes values by default, which helps prevent XSS (Cross-Site Scripting) attacks.

◆ **Common JSX Mistakes (Interview Favorite ❌)**

❌ **Using `class` instead of `className`**

```
<div class="box"></div> // Wrong
```



```
<div className="box"></div>
```

❌ **Forgetting to close tags**

```
 // Wrong
```



```

```

✗ Using **if-else** directly inside JSX

```
{ if(isLoggedIn) { return <Home/> } } // Wrong
```



```
{isLoggedIn && <Home />}
```

✗ Returning multiple elements without parent

```
return (  
  <h1>Hello</h1>  
  <p>World</p>  
);
```



```
return (  
  <>  
    <h1>Hello</h1>  
    <p>World</p>  
  </>  
);
```

◆ One-line JSX Definition (Very Important ★)

JSX is syntactic sugar for `React.createElement()` that allows writing HTML-like code in JavaScript.

◆ Hinglish Explanation (Easy to Remember)

JSX ek JavaScript ka extension hai jo hume JavaScript ke andar HTML jaisa code likhne deta hai. Browser JSX ko directly nahi samajhta, isliye Babel usse JavaScript me convert karta hai.

If you want next:

- 🔥 Virtual DOM explanation
- 🔥 Props vs State
- 🔥 `useState` & `useEffect` interview questions
- 🔥 React lifecycle

Just tell me what to prepare next 🚀

Yes, JSX escapes values by default, which helps prevent XSS (Cross-Site Scripting) attacks. explain this

This is an **important security concept**, and interviewers love it.
I'll explain it **step-by-step, with example + interview answer**.

What does “JSX escapes values by default” mean?

When you insert **dynamic data** into JSX using `{ }`,
React automatically converts dangerous characters into safe text instead of executing them as code.

👉 This prevents **XSS (Cross-Site Scripting) attacks**.

What is XSS (in simple words)?

XSS is an attack where a hacker injects **malicious JavaScript** into a webpage, like:

```
<script>alert("Hacked!")</script>
```

If this runs, the attacker can:

- Steal cookies
- Access user data
- Hijack sessions

How JSX protects us (Automatic Escaping)

Example  **User input (dangerous)**

```
const userInput = "<script>alert('Hacked')</script>";
```

JSX usage

```
<h1>{userInput}</h1>
```

What React does internally

React **escapes** it and renders:

```
&lt;script&gt;alert('Hacked')&lt;/script&gt;
```

 **Result on screen:**

```
<script>alert('Hacked')</script>
```

- ✓ Script is shown as **text**
 - ✗ Script is **NOT** executed
-

Why this happens?

Because:

- JSX treats everything inside `{ }` as **data**
- React sanitizes (escapes) it before rendering

This makes React apps **secure by default**.

What characters are escaped?

React escapes:

- `<` → `<`
- `>` → `>`
- `"` → `"`
- `'` → `'`

So browsers don't treat them as executable HTML or JS.

When React does NOT escape values ⚠

If you **explicitly** tell React to trust the content:

❌ Dangerous example

```
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

Now React:

- ❌ Does NOT escape
- ❌ Executes HTML/JS

That's why it's called **dangerously**.

📌 Interview tip:

Use `dangerouslySetInnerHTML` only when data is trusted (like sanitized content from backend).

Interview-ready explanation ★

JSX escapes values by default, meaning it converts special characters into safe text so that injected scripts are not executed. This protects React applications from XSS attacks. Only when using `dangerouslySetInnerHTML` does React skip escaping.

One-line answer (Perfect for quick round)

React prevents XSS by automatically escaping dynamic values rendered in JSX.

What is `dangerouslySetInnerHTML`?

`dangerouslySetInnerHTML` is a **React property** that allows you to **insert raw HTML directly into the DOM**.

📌 It tells React:

“Do NOT escape this content. Trust me.”

That’s why it is called **dangerously**.

Why does React need this?

By default, React **escapes HTML** to prevent XSS.

But sometimes you **must render HTML**, like:

- HTML content from CMS (WordPress, Strapi)
- Blog articles stored as HTML
- Markdown converted to HTML
- Email templates

In such cases, JSX alone cannot render HTML tags.

Syntax (Very Important)

```
<div dangerouslySetInnerHTML={{ __html: htmlContent }} />
```

 Notice:

- It takes an **object**
 - Key must be `__html` (double underscore)
 - Value is the HTML string
-

Example Without `dangerouslySetInnerHTML`

```
const htmlContent = "<h1>Hello</h1>";  
  
<div>{htmlContent}</div>
```

● Output on screen:

```
<h1>Hello</h1>
```

(Tag is treated as text)

Example 2 With `dangerouslySetInnerHTML`

```
<div dangerouslySetInnerHTML={{ __html: htmlContent }} />
```

● Output on screen:

```
Hello
```

(Tag is rendered as HTML)

Why is it dangerous? (XSS Explained)

Malicious input

```
const htmlContent = "<img src=x onerror=alert('Hacked')  
/>";
```

Rendering it

```
<div dangerouslySetInnerHTML={{ __html: htmlContent }} />
```

💥 Result:

- JavaScript executes
- App is vulnerable to XSS

This is exactly what React normally **prevents**.

Why React named it “dangerously”?

React intentionally chose this long name to:

- Make developers **pause and think**
- Prevent accidental usage
- Highlight **security risk**

 Interview line:

React uses the word “dangerously” to warn developers about potential XSS attacks.

When should you use it?

Use only when:

- Content comes from a **trusted source**
- HTML is **sanitized**
- You **cannot avoid** raw HTML

Common use cases:

- CMS content

- Rich text editor output
 - Markdown preview
-

When should you NOT use it? ❌

- User input
 - Forms
 - Chat messages
 - Comments section (without sanitization)
-

How to use it safely 🔒

1 Sanitize HTML before rendering

Use libraries like:

- `DOMPurify`

Example:

```
import DOMPurify from "dompurify";

const safeHTML = DOMPurify.sanitize(htmlContent);

<div dangerouslySetInnerHTML={{ __html: safeHTML }} />
```

Comparison: JSX vs dangerouslySetInnerHTML

Feature	<code>JSS</code>	<code>dangerouslySetInnerHTML</code>
	<code>{}</code>	
Escapes HTML		
Safe by default		
Allows raw HTML		
XSS risk		

Interview-ready answer

`dangerouslySetInnerHTML` is a React property used to render raw HTML directly into the DOM. React skips escaping content when using it, which can lead to XSS attacks if the HTML is not sanitized. Therefore, it should only be used with trusted and sanitized data.

4. Explain Virtual DOM and Real DOM

<https://dev.to/swarnaliroy94/introduction-to-react-real-dom-virtual-dom-363>

This is a **core React interview topic**.

I'll explain **Real DOM vs Virtual DOM** in a **simple, structured, interview-ready way**.

1 What is Real DOM?

Real DOM is the **actual DOM tree** created by the browser.

- Every HTML element becomes a node

- Directly represents the UI on the screen
- Any change updates the browser UI immediately

Example

```
document.getElementById("title").innerHTML = "Hello";
```

 Problem:

- Updating the Real DOM is **slow**
 - Even small changes can cause **re-rendering of large parts of UI**
-

2 What is Virtual DOM?

Virtual DOM is a **lightweight JavaScript object** that is a **copy of the Real DOM**.

- Stored in memory
- Created and managed by React
- Much faster to update

 React never directly updates the Real DOM first.

How React uses Virtual DOM (Step-by-step)

1. State or props change
2. React creates a **new Virtual DOM**

3. React compares it with the **previous Virtual DOM**
(This process is called **diffing**)
4. React finds the **minimum changes**
5. Only those changes are applied to the **Real DOM**

👉 This is called **Reconciliation**

Example (Easy to understand)

Initial UI

```
<h1>Hello</h1>
```

Updated UI

```
<h1>Hello Vishal</h1>
```

🔴 Without Virtual DOM:

- Entire DOM tree may re-render

🟢 With Virtual DOM:

- Only text inside `<h1>` is updated
-

Why Virtual DOM is faster?

Because:

- Updating JS objects is fast

- Real DOM manipulation is expensive
- React minimizes DOM updates

Real DOM vs Virtual DOM (Comparison Table)

Feature	Real DOM	Virtual DOM
Type	Browser DOM	JS Object
Update Speed	Slow	Fast
Direct Access	Yes	No
Memory Usage	Less	More
UI Re-render	Whole tree	Only changed nodes
Used by	Vanilla JS	React

Important Interview Points

- Virtual DOM is **not a real browser DOM**
- React updates **Virtual DOM first**
- Diffing algorithm finds minimal changes

- Improves performance in large applications
-

Common Interview Question

? Is Virtual DOM always faster?

Answer:

Not always. For very small apps, Real DOM may be faster. Virtual DOM shows benefits mainly in **large and complex UIs**.

Interview-ready answer (1 minute)

The Real DOM is the actual browser DOM that directly updates the UI, while the Virtual DOM is a lightweight JavaScript representation of the Real DOM used by React. React compares the previous and new Virtual DOM using diffing and updates only the changed parts of the Real DOM, improving performance.

One-line answer (Quick round)

Virtual DOM is an in-memory representation of the Real DOM that allows React to efficiently update only changed elements.

Hinglish Explanation

Real DOM browser ka actual DOM hota hai, jisme direct change karna slow hota hai. Virtual DOM ek JS copy hoti hai jisme React pehle changes karta hai aur sirf required updates hi Real DOM me apply karta hai.

React Hooks

1.useState()

2.useEffect()

What is `useEffect`?

`useEffect` is a React Hook used to perform side effects in functional components.

📌 Side effects are operations that are outside normal rendering, such as:

- Fetching data from an API
 - Updating the DOM
 - Timers (`setTimeout`, `setInterval`)
 - Subscriptions (events, sockets)
-

Why do we need `useEffect`?

In React:

- Rendering should be pure (no side effects)
- Side effects must run after render

👉 `useEffect` runs after the component renders.

Basic Syntax

```
useEffect(() => {  
  // side effect code  
  
  return () => {  
    // cleanup code (optional)  
  };  
}, [dependencies]);
```

How `useEffect` works (Important ★)

`useEffect` behavior depends on the dependency array.

1 No dependency array

```
useEffect(() => {  
  console.log("Runs on every render");  
});
```

📌 Runs:

- After every render
 - After every state/props change
-

2 Empty dependency array []

```
useEffect(() => {  
  console.log("Runs only once");
```

```
}, []);
```

 Runs:

- Once, after initial render
 - Similar to `componentDidMount`
-

③ With dependencies

```
useEffect(() => {  
  console.log("Runs when count changes");  
}, [count]);
```

 Runs:

- On initial render
 - Whenever `count` changes
-

Cleanup function (Very Important)

Used to prevent memory leaks.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running...");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
  };  
});
```

```
}, []);
```

📌 Cleanup runs:

- Before component unmounts
- Before effect re-runs

Lifecycle mapping (Interview Favorite)

Class Component	useEffect
componentDidMount	<code>useEffect(..., [])</code>
componentDidUpdate	<code>useEffect(..., [deps])</code>
componentWillUnmount	cleanup function

Real-world example (API call)

```
useEffect(() => {  
  fetch("/api/users")  
    .then(res => res.json())  
    .then(data => setUsers(data));  
}, []);
```

Rules of `useEffect` ★

1. Always use it inside a functional component

2. Do not use it inside loops or conditions
 3. Include all used variables in dependency array
-

Common mistakes ❌

- ❌ Missing dependency array
 - ❌ Infinite loops due to wrong dependencies
 - ❌ Forgetting cleanup
-

Interview-ready answer ★

useEffect is a React Hook used to handle side effects in functional components. It runs after rendering and can be controlled using a dependency array.

One-line answer (Quick round)

useEffect handles side effects like API calls and subscriptions in React functional components.

Hinglish explanation

useEffect React ka hook hai jo side effects jaise API call, timer, aur subscriptions handle karta hai aur render ke baad execute hota hai.

If you want next:

- 🔥 **useEffect** dependency array deep dive
- 🔥 Infinite loop explanation
- 🔥 **useEffect** vs **useLayoutEffect**
- 🔥 Interview Q&A on **useEffect**

Just tell me 🚀

explain this 📌 Side effects are operations that are outside normal rendering, such as:

Good question 👍

This line is key to understanding **useEffect**, so let's explain it very clearly and practically.

What does “outside normal rendering” mean?

Normal rendering = 👉

React's rendering job is only to:

- Take state + props
- Return JSX
- Show UI on screen

👉 Rendering should be pure:

- Same input → same UI
 - No external changes
-

Then what are Side Effects?

Side effects are operations that:

- Do something extra
- Affect the outside world
- Are not just UI rendering

That's why they are called outside normal rendering.

Side Effects Explained One by One

1 Fetching data from an API 🌐

```
fetch("/users")
```

Why is this a side effect?

- It talks to a server
- Takes time
- Changes app state later

👉 Not part of rendering UI directly.

2 Updating the DOM manually 🖥️

```
document.title = "Home Page";
```

Why side effect?

- Changes browser environment
 - Happens outside React's render process
-

3 Timers (`setTimeout`, `setInterval`)

```
setInterval(() => {  
  console.log("Running");  
}, 1000);
```

Why side effect?

- Runs continuously
 - Needs cleanup
 - Not related to JSX return
-

4 Subscriptions (events, sockets)

```
window.addEventListener("resize", handleResize);
```

Why side effect?

- Listens to external events
 - Must be cleaned up
 - Affects app behavior
-

5 Local storage / session storage

```
localStorage.setItem("token", token);
```

Why side effect?

- Writes to browser storage
 - External to rendering
-

Why React separates side effects from rendering?

Because:

- Rendering must be fast
- Rendering must be predictable
- Side effects can be slow or unsafe

👉 So React says:

“Render first, then run side effects”

That’s exactly what `useEffect` does.

Simple mental model 🧠

Render phase

`State` → `JSX` → `UI`

Effect phase

`API calls, timers, subscriptions`

Example (Very Clear)

✗ Wrong (side effect during render)

```
function App() {  
  document.title = "Home"; // ✗ side effect in render  
  return <h1>Home</h1>;  
}
```

✓ Correct

```
function App() {  
  useEffect(() => {  
    document.title = "Home";  
  }, []);  
  
  return <h1>Home</h1>;  
}
```

Interview-ready explanation ★

Side effects are operations that interact with the outside world, such as API calls, timers, DOM updates, or subscriptions. These operations are not part of pure rendering and are handled using `useEffect`.

4.useRef()

What is `useRef`?

`useRef` is a **React Hook** that allows you to:

1. **Access DOM elements directly**
2. **Store mutable values** that do NOT cause re-render when changed

Syntax

```
const myRef = useRef(initialValue);
```

- Returns an object: `{ current: initialValue }`
- Value is stored in `myRef.current`

Use Case **1** Accessing DOM elements (Most common)

Example

```
function App() {  
  const inputRef = useRef(null);  
  
  const focusInput = () => {  
    inputRef.current.focus();  
  };  
  
  return (  
    <>  
      <input ref={inputRef} />  
      <button onClick={focusInput}>Focus</button>  
    </>  
  );  
}
```

 Why use `useRef`?

- No `document.getElementById`

- Works with React's lifecycle
 - Cleaner & safer
-

Use Case 2 Storing values without re-render

```
function Counter() {  
  const countRef = useRef(0);  
  
  const increase = () => {  
    countRef.current += 1;  
    console.log(countRef.current);  
  };  
  
  return <button onClick={increase}>Increase</button>;  
}
```

📌 Here:

- Value changes
 - Component **does NOT re-render**
-

Difference: **useRef** vs **useState** ★

useRef	useState
Does NOT cause re-render	Causes re-render
Mutable value	Immutable state

Access DOM

Manage UI
data

`.current` property

Setter function

When should you use `useRef`?

- Focusing inputs
 - Measuring DOM size
 - Storing timers / intervals
 - Keeping previous values
 - Avoiding unnecessary re-renders
-

When NOT to use `useRef`

- For displaying data in UI
 - For state that affects rendering
-

Common Interview Question

Q: Does changing `ref.current` cause re-render?

 No

Interview-ready answer 

`useRef` is a React Hook that provides a mutable reference which persists across renders and can be used to access DOM elements or store values without triggering re-renders.